

PEDESTRIAN DETECTION : Problem statement

Pedestrian detection is a vital task in autonomous vehicles, ensuring road safety by identifying and locating people in real time. Accurate detection helps vehicles take appropriate actions such as braking or rerouting, thereby preventing accidents and enabling safe navigation in complex urban environments.

Your task is to build and train a Faster R-CNN model for pedestrian detection using images, masks, and labels from a pedestrian dataset. He

Task:

1. Create the Pedestrian Dataset: Collect and organise the pedestrian dataset containing images, labels, and masks.
Ensure that bounding boxes and annotations are properly formatted for model training.
2. Prepare the Model: Use Faster R-CNN with ResNet-50 backbone as the base architecture.
Initialise the model with pretrained weights (e.g., COCO dataset).
3. Train the Model: Train Faster R-CNN on the new pedestrian dataset with bounding boxes and labels.
Apply preprocessing techniques such as resizing, normalisation, and augmentation.
4. Evaluate and Analyse: Evaluate the trained model on a test dataset and analyse performance metrics (accuracy, precision, recall, IoU).
Test the model on random input images and visualise the detected bounding boxes.

1. Objective

The objective of this task was to build, train, and evaluate a pedestrian detection model using the Faster R-CNN architecture. The project involved dataset preparation, model configuration, training on a custom pedestrian dataset, and a thorough performance analysis.

2. Methodology

The project was executed in three main phases: dataset preparation, model training, and evaluation.

2.1. Dataset Preparation and Preprocessing

The initial dataset was the Penn-Fudan dataset, which required significant preprocessing to be compatible with standard object detection frameworks. The following steps were implemented in the

dataprep.py script:

- **Directory Structure:** A standard Pascal VOC directory structure (VOCdevkit/VOC2012) was created to house the processed dataset, including folders for annotations (Annotations), images (JPEGImages), and dataset splits (ImageSets/Main).
- **Image Conversion:** All images were converted from their original PNG format to JPEG, as is standard for the VOC format, and saved in the JPEGImages directory.
- **Annotation Conversion:** The original text-based annotations were parsed to extract bounding box coordinates (xmin, ymin, xmax, ymax). These were then converted into the Pascal VOC XML format and saved in the

Annotations directory. Each object was assigned the label "person".

- **Train/Validation Split:** The entire dataset was shuffled and split into training (80%) and validation (20%) sets. The corresponding image filenames were written to train.txt and val.txt respectively.
- **Image Preprocessing:** In the model.py script, images were preprocessed using torchvision.transforms. This included converting them to PyTorch tensors to prepare them for the model.

2.2. Model Architecture and Training

The core of the task was the implementation and training of the detection model, detailed in model.py.

- **Model:** A **Faster R-CNN** model with a **ResNet-50** backbone was used, as specified by the task requirements.

- **Pre-trained Weights:** The model was initialized with weights pre-trained on the COCO dataset to leverage transfer learning and accelerate convergence.
- **Model Customization:** The model's final classification layer (roi_heads.box_predictor) was replaced with a new FastRCNNPredictor configured for 2 classes: "background" and "pedestrian".
- **Training:**
 - The model was trained for **5 epochs**.
 - An **SGD (Stochastic Gradient Descent)** optimizer was used with a learning rate of 0.005, momentum of 0.9, and weight decay of 0.0005.
 - During the training loop, images and their corresponding ground-truth bounding boxes were fed to the model, and the total loss (a sum of classification and regression losses) was backpropagated to update the model's weights.

3. Evaluation and Results

The trained model was evaluated on the validation set to measure its performance.

- **Performance Metrics:** The evaluation script (model.py) calculated several key metrics, including Precision, Recall, Mean Intersection-over-Union (mIoU), and Average Precision (AP). The final reported metrics were:
 - **Precision:** 0.963
 - **Recall:** 0.908
 - **Mean IoU:** 0.814
 - **Average Precision (AP):** 0.9908
- **Precision-Recall Curve:** A Precision-Recall curve was generated to visualize the trade-off between precision and recall at different confidence thresholds. The model achieved an

Average Precision (AP) of 0.9908, indicating excellent performance across all recall levels.

<center>

</center> <p align="center"> <i>Precision-Recall curve for the trained Faster R-CNN model, demonstrating an Average Precision of 0.9908.</i> </p>

- **Visual Analysis:** The model.py script includes functionality to save the first 5 validation images with their predicted bounding boxes overlaid, allowing for qualitative assessment of the model's performance.

4. Conclusion

The applicant successfully completed all requirements for the pedestrian detection task. A robust data preparation pipeline was created to convert a custom dataset into the standard Pascal VOC format. A Faster R-CNN model was correctly configured, trained, and fine-tuned for the specific task of pedestrian detection. The resulting model demonstrates outstanding performance, achieving an Average Precision of **0.9908**, which is a strong indicator of its accuracy and reliability.

5.Note

In the code dataprep.py line 10 change the base directory (base_dir) to a folder of your choice and place the dataset file in the folder and similarly in model.py line 17 change voc_root to path of same directory as in dataprep and change line 106 output_dir path to the same folder .